

อธิบายโค้ดโครงสร้างพื้นฐานของโปรเจกต์

ส่วน import โมดูลที่จำเป็น

```
from heapq import heappush, heappop
from collections import deque
import math
import tkinter as tk
from tkinter import simpledialog, messagebox, filedialog
import json
```

- heapq (heappush, heappop)
 - ใช้สำหรับสร้าง min-heap หรือ priority queue
 - อัลกอริทึม Dijkstra และ Prim ใช้ heap เพื่อเลือกเส้นที่มีน้ำหนักน้อยที่สุดอย่างรวดเร็ว
- collections.deque
 - ใช้สร้าง queue แบบเร็วสำหรับ BFS
 - append() = ใสท้ายคิว
 - popleft() = ดึงจากหัวคิว
- math
 - ใช้ฟังก์ชัน math.inf สำหรับแทนค่าระยะที่ “ยังไม่เคยรู้”
- tkinter และโมดูลย่อย
 - ใช้สร้าง GUI ทั้งหมด เช่น ปุ่ม, หน้าต่างข้อความ, กล่องรับอินพุต, ระบบโหลด/บันทึกไฟล์
- json
 - ใช้เก็บ-โหลดข้อมูลกราฟลงไฟล์ .json (ในพีเจอร์ Save Graph / Load Graph)

ตัวแปรพื้นฐานสำหรับการจัดการ label ของจุด

```
INDEX_BASE = 1

label_to_index = {}
index_to_label = {}
```

- INDEX_BASE = 1
 - จุด A (หรือจุดแรก) จะมี index เป็น 1 แทนการเริ่มจาก 0 ตามแบบคณิตศาสตร์มนุษย์อ่านง่ายขึ้น
 - ในโค้ดจริง index ยังเป็น 0-based แต่เวลาแสดงผลจะแสดงแบบ 1, 2, 3,...
- label_to_index
 - เป็น dictionary สำหรับแปลงชื่อจุด → เลข index
 - เช่น "A" → 0
- index_to_label
 - แปลงกลับจากเลข index → label
 - เช่น 0 → "A"

อธิบายโค้ดโครงสร้างพื้นฐานของโปรเจกต์

ฟังก์ชันแปลงเลข index เป็นชื่อจุด

```
def index_to_letters(i: int) -> str:
    """แปลง 0 -> 1, 1 -> 2, ..."""
    return str(i + 1)
```

- ฟังก์ชันนี้รับ index แบบ 0-based แต่แสดงผลเป็น “1-based”
เช่น
 - index 0 → "1"
 - index 1 → "2"
 - index 2 → "3"

ฟังก์ชันจัดรูปแบบชื่อจุด

```
def format_vertex(i: int) -> str:
    return index_to_letters(i)
```

- เป็นตัวช่วยอีกชั้นหนึ่งเพื่อให้โค้ดอ่านง่าย
 - ใช้เวลาแสดงชื่อจุดในผลลัพธ์หรือใน UI เช่น BFS order, Dijkstra path ฯลฯ

ฟังก์ชันแปลงข้อความเป็น index ของจุด

```
def parse_vertex_token(tok: str) -> int:
    tok = tok.strip()
    if tok.isdigit():
        return int(tok) - INDEX_BASE
    if tok in label_to_index:
        return label_to_index[tok]
    print("Unknown vertex label:", tok)
    raise SystemExit
```

อธิบายโค้ดโครงสร้างพื้นฐานของโปรเจกต์

ฟังก์ชันแปลงข้อความเป็น index ของจุด

```
def parse_vertex_token(tok: str) -> int:
    tok = tok.strip()
    if tok.isdigit():
        return int(tok) - INDEX_BASE
    if tok in label_to_index:
        return label_to_index[tok]
    print("Unknown vertex label:", tok)
    raise SystemExit
```

- tok.strip()
 - ล้างช่องว่างซ้ายขวา เช่น " 3 " → "3"
- tok.isdigit()
 - ถ้าเป็นตัวเลข เช่น "1", "2", "5"
- return int(tok) - INDEX_BASE
 - แปลงเป็น index แบบ 0-based
 - ผู้ใช้ใส่ "3" → index จริง = 3 - 1 = 2
- ถ้าชื่อเป็นตัวอักษร เช่น "A", "B"
 - จะหาจาก dict label_to_index
- ถ้าไม่พบ label ใดเลย → แจ้ง error แล้วหยุดโปรแกรม

ฟังก์ชันนี้รองรับกรณีผู้ใช้พิมพ์ชื่อจุดเอง แต่ใน GUI จะไม่ถูกเรียกใช้มากแล้ว

คลาสข้อมูลหลักของกราฟ

```
class Graph:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.edges = []

    def add_edge(self, u, v, w):
        self.adj[u].append((v, w))
        self.adj[v].append((u, w))
        self.edges.append((w, u, v))
        self.edges.append((w, v, u))
```

อธิบายโค้ดโครงสร้างพื้นฐานของโปรเจกต์

คลาสข้อมูลหลักของกราฟ

```
class Graph:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.edges = []
```

- class Graph:
 - ประกาศคลาสชื่อ Graph ใช้แทนโครงสร้างกราฟที่โปรแกรมทั้งระบบใช้ร่วมกัน
- def __init__(self, n):
 - ฟังก์ชันสร้างกราฟจำนวน n จุด
- self.n = n
 - เก็บจำนวนจุดทั้งหมดของกราฟ
- self.adj = [[] for _ in range(n)]
 - สร้าง adjacency list
 - เช่นถ้า $n = 4 \rightarrow$ `[[], [], [], []]` แต่ละตำแหน่งสำหรับใส่รายการเส้น เช่น `(v, w)`
- self.edges = []
 - เป็นลิสต์เก็บเส้นทั้งหมดในรูป `(w, u, v)` ใช้ใน Kruskal (เพราะต้อง sort ตามน้ำหนัก)

เมธอดสำหรับเพิ่มเส้น (Edge) ในกราฟ

```
def add_edge(self, u, v, w):
    self.adj[u].append((v, w))
    self.adj[v].append((u, w))
    self.edges.append((w, u, v))
    self.edges.append((w, v, u))
```

- self.adj[u].append((v, w))
 - เพิ่มข้อมูลเส้นจากจุด u ไป v น้ำหนัก w
- self.adj[v].append((u, w))
 - กราฟนี้เป็น undirected graph จึงต้องเพิ่มเส้น “กลับด้าน” ด้วย
- self.edges.append((w, u, v))
 - เพิ่มรูปแบบข้อมูลแบบ tuple `(w, u, v)` ใช้สำหรับ Kruskal
- self.edges.append((w, v, u))
 - เพิ่มเส้นกลับด้านเพื่อให้ Kruskal เห็นครบ

Depth-First Search (DFS) Spanning Tree

```
# ----- DFS -----
def dfs_spanning_tree(g, start):
    visited = [False] * g.n
    tree = []

    def dfs(u):
        visited[u] = True
        for v, w in g.adj[u]:
            if not visited[v]:
                tree.append((u, v, w))
                dfs(v)

    dfs(start)
    unreachable = [i for i in range(g.n) if not visited[i]]
    return tree, unreachable
```

แนวคิดโดยย่อ

DFS เป็นอัลกอริทึมที่ “เดินลึกไปให้สุดก่อนค่อยย้อนกลับ” ในกราฟในที่นี้ฟังก์ชัน `dfs_spanning_tree` จะสร้าง ต้นไม้แผ่ (Spanning Tree) จากจุดเริ่มต้น `start` และเก็บรายชื่อจุดที่ไม่ถึง (`unreachable`).

โค้ดทำงานอย่างไร

- **`def dfs_spanning_tree(g, start)`** : ประกาศฟังก์ชันชื่อ `dfs_spanning_tree` `g` คือกราฟ (มีข้อมูลจำนวนจุด `g.n` และ adjacency list `g.adj`) `start` คือ index ของจุดเริ่มต้น (0,1,2,...)
- **`visited = [False] * g.n`** สร้างลิสต์ `visited` ขนาดเท่าจำนวนจุดในกราฟ (`g.n`) ใช้เก็บสถานะว่า “เคยเยี่ยมชมจุดนี้แล้วหรือยัง” (ยัง = `False`, เคยแล้ว = `True`)
- **`tree = []`** สร้างลิสต์เปล่าไว้เก็บข้อมูลเส้นในต้นไม้แผ่ (spanning tree) แต่ละสมาชิกจะเป็นทูเพิล (`u, v, w`) = เส้นจากจุด `u` ไป `v` มีน้ำหนัก `w`
- **`def dfs(u)`**: ประกาศฟังก์ชันย่อย `dfs` (recursive) สำหรับเดิน DFS เริ่มจากจุด `u`
- **`visited[u] = True`** ทำเครื่องหมายว่าจุด `u` ถูกเยี่ยมชมแล้ว
- **`for v, w in g.adj[u]`**: วนดู ทุกเพื่อนบ้าน ของจุด `u` **`g.adj[u]`** คือรายชื่อ (`v, w`) ที่เป็นเส้นจาก `u` → `v` น้ำหนัก `w`
- **`if not visited[v]`** : ถ้าจุด `v` ยังไม่เคยถูกเยี่ยมชม
- **`tree.append((u, v, w))`** เพิ่มเส้น (`u, v, w`) เข้าไปในลิสต์ `tree` แสดงว่าเส้นนี้เป็นส่วนหนึ่งของ DFS tree
- **`dfs(v)`** เรียกฟังก์ชันตัวเองแบบ recursive เพื่อเดินต่อไปที่จุด `v` (เจาะลึกลงไปเรื่อย ๆ)
- **`dfs(start)`** เริ่มเดิน DFS จากจุดเริ่มต้นที่กำหนด (`start`)
- **`unreach = [i for i in range(g.n) if not visited[i]]`** ใช้ list comprehension รวม ดัชนีของจุดที่ `visited = False` แปลว่าจุดเหล่านี้ “ไปไม่ถึง” จากจุดเริ่ม `start`
- **`return tree, unreachable`** คืนค่า: `tree` : รายชื่อเส้นใน DFS spanning tree `unreach` : รายชื่อจุดที่ไม่สามารถไปถึงได้จากจุดเริ่ม

Breadth-First Search (BFS) Spanning Tree

```
# ----- BFS -----
def bfs_spanning_tree(g, start):
    visited = [False] * g.n
    tree = []
    q = deque()

    visited[start] = True
    q.append(start)

    while q:
        u = q.popleft()
        for v, w in g.adj[u]:
            if not visited[v]:
                visited[v] = True
                tree.append((u, v, w))
                q.append(v)

    unreach = [i for i in range(g.n) if not visited[i]]
    return tree, unreach
```

แนวคิดโดยย่อ

BFS เป็นการค้นหาแบบ “กว้างก่อน” ใช้คิว (queue) ช่วย ฟังก์ชัน bfs_spanning_tree จะสร้างต้นไม้แผ่จากจุด start โดยเดินทีละชั้น (level)

โค้ดทำงานยังไง

- **def bfs_spanning_tree(g, start):** ประกาศฟังก์ชันสำหรับสร้าง BFS spanning tree g คือกราฟ (มีจำนวนจุด g.n และ adjacency list g.adj) start คือ index ของจุดเริ่มต้น
- **visited = [False] * g.n** สร้างลิสต์ visited ขนาดเท่าจำนวนจุดในกราฟ ใช้เก็บสถานะว่า “เคยเยี่ยมชมจุดนั้นแล้วหรือยัง”
- **tree = []** ลิสต์เปล่า สำหรับเก็บเส้นในต้นไม้ที่เกิดจาก BFS แต่ละสมาชิกอยู่ในรูป (u, v, w) = เส้นจาก u → v น้ำหนัก w
- **q = deque()** สร้างคิวว่าง ๆ โดยใช้ deque (queue แบบ FIFO) สำหรับเก็บลำดับจุดที่รอการขยาย
- **visited[start] = True** ทำเครื่องหมายว่าจุดเริ่มต้นถูกเยี่ยมชมแล้ว
- **q.append(start)** นำจุดเริ่มต้นใส่คิว เป็นจุดแรกที่รอการขยาย
- **while q:** วนลูปตราบใดที่คิวไม่ว่าง (ยังมีจุดรอขยายอยู่)
- **u = q.popleft()** ดึงจุดตัวหน้าสุดออกจากคิว (ตามหลัก First-In First-Out) ตัวแปร u คือจุดที่เราจะนำไปขยายหาจุดข้างเคียง
- **for v, w in g.adj[u]:** วนดูเพื่อนบ้านทุกตัวของ u v คือจุดปลายของเส้น w คือค่าน้ำหนักของเส้น u → v
- **if not visited[v]:** ถ้าจุด v ยังไม่เคยถูกเยี่ยมชมมาก่อน
- **visited[v] = True** ทำเครื่องหมายว่าจุด v เยี่ยมชมแล้ว
- **tree.append((u, v, w))** เพิ่มเส้น (u, v, w) ลงในลิสต์ tree แสดงว่าเส้นนี้เป็นส่วนหนึ่งของ BFS spanning tree (เพราะใช้เส้นนี้ในการค้นพบจุด v)
- **q.append(v)** นำจุด v เข้า queue เพื่อรอการขยายต่อไปในรอบถัด ๆ ไป
- **unreach = [i for i in range(g.n) if not visited[i]]** ใช้ list comprehension รวบรวม index ของทุกจุดที่ visited ยังเป็น False แปลว่าจุดเหล่านั้น “ไปไม่ถึง” จากจุดเริ่ม start
- **return tree, unreach** คืนค่า tree : รายการเส้นใน BFS spanning tree unreach : รายการดัชนีของจุดที่ไม่สามารถเดินทางไปถึงได้จาก start

Dijkstra's Shortest Path Algorithm

```
# ----- Dijkstra -----  
def dijkstra(g, src, dest):  
    dist = [math.inf] * g.n  
    parent = [-1] * g.n  
  
    dist[src] = 0  
    heap = [(0, src)]  
  
    while heap:  
        d, u = heappop(heap)  
        if d != dist[u]:  
            continue  
        if u == dest:  
            break
```

แนวคิดโดยย่อ

Dijkstra เป็นอัลกอริทึมสำหรับหา “เส้นทางที่มีน้ำหนักรวมน้อยที่สุด (shortest path)” จากจุดต้นทางหนึ่งไปยังจุดอื่น ๆ ในกราฟที่มีน้ำหนักเส้นเป็นค่าบวก ในที่นี้ฟังก์ชัน dijkstra ใช้หาเส้นทางสั้นที่สุดจากจุดเริ่ม src ไปยังจุดปลาย dest พร้อมคืนค่าระยะทางรวม และลำดับจุดบนเส้นทางนั้น

โค้ดทำงานยังไง

- **def dijkstra(g, src, dest):** ประกาศฟังก์ชัน Dijkstra g คือกราฟ src คือ index จุดต้นทาง dest คือ index จุดปลายทางที่ต้องการหา shortest path
- **dist = [math.inf] * g.n** สร้างลิสต์ dist เพื่อเก็บระยะทางที่ดีที่สุดจาก src ไปยังแต่ละจุด เริ่มต้นให้ทุกตำแหน่งเป็น ∞ (ยังไม่รู้ระยะทาง)
- **parent = [-1] * g.n** ลิสต์ parent ใช้บันทึกว่า เส้นทางที่ดีที่สุดของแต่ละจุด “มาจากจุดไหน” เพื่อใช้ย้อนรอยสร้างเส้นทางในภายหลัง
- **dist[src] = 0** ตั้งค่าระยะทางจาก src ไปตัวเองให้เป็น 0
- **heap = [(0, src)]** สร้าง priority queue แบบ min-heap โดยเก็บทูเพิล (ระยะทาง, จุด) เริ่มต้นด้วย (0, src) เป็น entry แรกใน heap
- **while heap:** วงลูปตรรกะใดที่ heap ยังมีข้อมูลอยู่
- **d, u = heappop(heap)** ดึงคู่ (d, u) ที่มีระยะทาง d น้อยที่สุดออกมาจาก heap u คือจุดที่กำลังพิจารณาขยายไปยังเพื่อนบ้าน d คือระยะทางปัจจุบันจาก src มาถึง u ตามข้อมูลใน heap
- **if d != dist[u]:** ถ้า d ไม่เท่ากับค่า dist[u] ปัจจุบัน แสดงว่า entry นี้ “เก่า” (ระหว่างทางเคยมีการอัปเดตระยะทางที่สั้นกว่านี้แล้ว) จึงไม่ต้องใช้
- **continue** ข้าม entry นี้ไปดูรายการถัดไปใน heap
- **if u == dest:** ถ้าจุดที่ดึงออกมาจาก heap เป็นจุดปลาย dest
- **break** สามารถหยุดลูปได้ เนื่องจากในอัลกอริทึม Dijkstra เมื่อ dest ถูกดึงออกมาจาก heap ครั้งแรก ระยะทางนั้นเป็น “สั้นที่สุดแน่นอน”

Dijkstra's Shortest Path Algorithm

```
for v, w in g.adj[u]:
    nd = d + w
    if nd < dist[v]:
        dist[v] = nd
        parent[v] = u
        heappush(heap, (nd, v))

if dist[dest] == math.inf:
    return math.inf, []

path = []
cur = dest
while cur != -1:
    path.append(cur)
    cur = parent[cur]
path.reverse()

return dist[dest], path
```

โค้ดทำงานยังไง (ต่อ)

- **for v, w in g.adj[u]:** วนดูเพื่อนบ้านทุกตัวของ u v คือ index จุดปลาย w คือค่าน้ำหนักของเส้น u → v
- **nd = d + w** คำนวณระยะทางใหม่ nd โดยสมมติว่าเดินจาก src → ... → u → v เท่ากับระยะทางเดิม d บวกน้ำหนักเส้น w
- **if nd < dist[v]:** ถ้าระยะทางใหม่นี้สั้นกว่าระยะทางที่เคยเก็บไว้ใน dist[v]
- **dist[v] = nd** อัปเดตระยะทางที่ดีที่สุดของ v ให้เป็นค่าที่ดีกว่า
- **parent[v] = u** บันทึกว่าทางที่ดีที่สุดมาถึง v ในตอนนี้ มาจากจุด u (ใช้สร้างเส้นทางย้อนกลับในภายหลัง)
- **heappush(heap, (nd, v))** นำ (nd, v) เข้า heap เพื่อเป็นตัวเลือกให้ดึงมาขยายในรอบถัด ๆ ไป
- **if dist[dest] == math.inf:** หลังจบลูป ถ้าระยะทางของ dest ยังเป็น ∞ อยู่
- **return math.inf, []** แสดงว่าไม่มีเส้นทางจาก src ไป dest เลยคืนระยะทางเป็น ∞ และคืน path ว่าง
- **path = []** เตรียมลิสต์ว่างสำหรับเก็บลำดับจุดบนเส้นทางที่สั้นที่สุด
- **cur = dest** เริ่มย้อนรอยจากจุดปลาย dest
- **while cur != -1:** วนย้อนขึ้นไปเรื่อย ๆ จนกระทั่งไม่มีพ่อ (parent = -1)
- **path.append(cur)** เพิ่มจุดปัจจุบัน cur ลงในลิสต์ path
- **cur = parent[cur]** ขยับไปที่จุดพ่อของ cur ตามข้อมูลใน parent
- **path.reverse()** กลับลำดับลิสต์ เพราะตอนที่ย้อนรอย เราไล่จาก dest → src เมื่อลองกลับลำดับจะได้เส้นทางจาก src → ... → dest ตามลำดับที่ถูกต้อง
- **return dist[dest], path** คืนค่า ระยะทางสั้นที่สุดจาก src ไป dest ลิสต์ลำดับจุดบนเส้นทางนั้น

Prim's Minimum Spanning Tree (MST)

```
# ----- Prim -----  
def prim(g, start):  
    visited = [False] * g.n  
    visited[start] = True  
    heap = []  
  
    for v, w in g.adj[start]:  
        heappush(heap, (w, start, v))  
  
    mst = []  
    total = 0  
  
    while heap and len(mst) < g.n - 1:  
        w, u, v = heappop(heap)  
        if visited[v]:  
            continue
```

แนวคิดโดยย่อ

Prim เป็นอัลกอริทึมสำหรับสร้าง Minimum Spanning Tree (MST) จากกราฟที่มีน้ำหนักเส้นเป็นค่าบวก โดยเริ่มจากจุดใดจุดหนึ่ง แล้วค่อย ๆ เลือก “เส้นที่มีน้ำหนักต่ำที่สุด” ที่เชื่อมจากกลุ่มโหนดที่อยู่ในต้นไม้ไปยังโหนดใหม่ที่ยังไม่อยู่ในต้นไม้ จนกว่าจะครอบคลุมทุกโหนดหรือไม่สามารถต่อได้อีก

โค้ดทำงานอย่างไร

- **def prim(g, start):** ประกาศฟังก์ชัน Prim g คือกราฟ start คือ index ของจุดเริ่มต้นสำหรับสร้าง MST
- **visited = [False] * g.n** ลิสต์ visited สำหรับเก็บสถานะว่าโหนดแต่ละตัวถูกเพิ่มเข้า MST แล้วหรือยัง
- **visited[start] = True** เริ่มต้นด้วยการนำโหนด start เข้าต้นไม้ MST เป็นโหนดตัวแรก
- **heap = []** สร้าง min-heap สำหรับเก็บเส้นขอบ (edges) ที่เชื่อมจากโหนดใน MST ไปยังโหนดนอก MST แต่ละ entry จะอยู่ในรูป (w, u, v) คือน้ำหนัก w จาก u → v
- **for v, w in g.adj[start]:** วนดูเพื่อนบ้านทุกตัวของจุดเริ่มต้น start
- **heappush(heap, (w, start, v))** เพิ่มเส้นจาก start → v เข้า heap พร้อมน้ำหนัก w เพื่อให้ heap สามารถเลือกเส้นที่เบาที่สุดได้ในขั้นถัดไป
- **mst = []** ลิสต์เก็บรายการเส้นที่ถูกเลือกเข้ามาเป็นส่วนหนึ่งของ MST
- **total = 0** ตัวแปรเก็บผลรวมของน้ำหนักเส้นทั้งหมดใน MST
- **while heap and len(mst) < g.n - 1:** วนลูปตราบใดที่ยังมีเส้นใน heap และจำนวนเส้นใน mst ยังน้อยกว่า จำนวนโหนด - 1 (MST ที่สมบูรณ์จะต้องมีเส้นเท่ากับ n - 1 เส้น)
- **w, u, v = heappop(heap)** ดึงเส้นที่มีน้ำหนักน้อยที่สุดออกจาก heap u คือโหนดต้นทาง v คือโหนดปลายทาง w คือค่าน้ำหนักเส้น
- **if visited[v]:** ถ้าโหนด v เคยถูกเพิ่มเข้า MST แล้ว
- **continue** ข้ามเส้นนี้ เพราะจะทำให้วนกลับซ้ำในต้นไม้ (ไม่จำเป็นและอาจสร้าง cycle)

Prim's Minimum Spanning Tree (MST)

```
visited[v] = True
mst.append((u, v, w))
total += w

for to, ww in g.adj[v]:
    if not visited[to]:
        heappush(heap, (ww, v, to))

if len(mst) != g.n - 1:
    return None, math.inf
return mst, total
```

โค้ดทำงานยังไง (ต่อ)

- **visited[v] = True** ทำเครื่องหมายว่าโหนด v ถูกเพิ่มเข้า MST แล้ว
- **mst.append((u, v, w))** เพิ่มเส้น (u, v, w) ลงในลิสต์ mst เป็นส่วนหนึ่งของ MST
- **total += w** บวกน้ำหนักเส้นนี้เข้าไปในผลรวมของ MST
- **for to, ww in g.adj[v]:** วนดูเพื่อนบ้านทุกตัวของ v
- **if not visited[to]:** ถ้าโหนดปลาย to ยังไม่ได้ถูกเพิ่มเข้า MST
- **heappush(heap, (ww, v, to))** เพิ่มเส้น $v \rightarrow to$ พร้อมน้ำหนัก ww เข้า heap เพื่อใช้เป็นตัวเลือกในการขยาย MST ในรอบถัดไป
- **if len(mst) != g.n - 1:** หลังจบลูป ถ้าจำนวนเส้นใน MST ไม่ถึง $n - 1$
- **return None, math.inf** แสดงว่ากราฟไม่เชื่อมถึงกันทุกโหนด (ไม่สามารถสร้าง MST ครอบคลุมทุกโหนดได้)
- **return mst, total** มิฉะนั้นคืนค่า mst : รายการเส้นใน Minimum Spanning Tree total : ผลรวมน้ำหนักของเส้นทั้งหมดใน MST

Kruskal's Minimum Spanning Tree (MST)

```
# ----- Kruskal -----
def kruskal(g):
    parent = [i for i in range(g.n)]
    rank = [0] * g.n

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
```

แนวคิดโดยย่อ

Kruskal เป็นอัลกอริทึมสำหรับสร้าง MST โดยใช้แนวคิด “เลือกเส้นที่เบาที่สุดก่อน” จากกราฟทั้งหมด แล้วเลือกเส้นเหล่านั้นทีละเส้น แต่ต้องระวังไม่ให้เกิดวงจร (cycle) ภายในต้นไม้ จึงใช้โครงสร้าง Disjoint Set Union (DSU) หรือ Union-Find ในการตรวจว่าการเพิ่มเส้นหนึ่ง ๆ ทำให้โหนดปลายทั้งสองอยู่ในกลุ่มเดียวกันอยู่แล้วหรือไม่

โค้ดทำงานยังไง

- **def kruskal(g):** ประกาศฟังก์ชัน Kruskal รับกราฟ g เป็นข้อมูลเข้า
- **parent = [i for i in range(g.n)]** สร้างลิสต์ parent สำหรับโครงสร้าง DSU เริ่มต้นให้แต่ละโหนดเป็น “หัวหน้ากลุ่มของตัวเอง” ($\text{parent}[i] = i$)
- **rank = [0] * g.n** สร้างลิสต์ rank สำหรับเก็บระดับความสูงโดยประมาณของแต่ละกลุ่ม ใช้ในการทำ union-by-rank เพื่อลดความสูงของต้นไม้ใน DSU
- ฟังก์ชันย่อย **def find(x):** ฟังก์ชันหา “รากกลุ่ม” (root) ของสมาชิก x ใน DSU
- **if parent[x] != x:** ถ้า x ไม่ใช่รากของตัวเอง
- **parent[x] = find(parent[x])** เรียก find ซ้ำเพื่อหาตัวรากที่แท้จริงของกลุ่ม และใช้เทคนิค path compression โดยบันทึก parent[x] ให้ชี้ตรงไปที่รากทันที ทำให้การค้นหาในครั้งต่อไป เร็วขึ้น
- **return parent[x]** คืนค่า index ของรากกลุ่มที่ x สังกัดอยู่
- ฟังก์ชันย่อย **def union(a, b):** ฟังก์ชันสำหรับรวมสองกลุ่มที่มีสมาชิก a กับ b ให้เป็นกลุ่มเดียวกัน
- **ra, rb = find(a), find(b)** หา “รากกลุ่ม” ของ a และ b ตามลำดับ
- **if ra == rb:** ถ้าทั้งสองมีรากกลุ่มเดียวกันอยู่แล้ว หมายความว่า a และ b อยู่ในคอมโพเนนต์เดียวกันตั้งแต่ก่อนรวม
- **return False** ไม่ควรรวมกลุ่มซ้ำ เพราะจะทำให้เกิดวงจร (cycle) หากใช้ใน MST ฟังก์ชันจึงคืนค่า False เพื่อบอกว่า union ครั้งนี้ “ไม่สำเร็จ/ไม่ควรใช้เส้นนี้”

Kruskal's Minimum Spanning Tree (MST)

```
def union(ra, rb):
    if rank[ra] < rank[rb]:
        parent[ra] = rb
    elif rank[ra] > rank[rb]:
        parent[rb] = ra
    else:
        parent[rb] = ra
        rank[ra] += 1
    return True

mst = []
total = 0

for w, u, v in sorted(g.edges):
    if union(u, v):
        mst.append((u, v, w))
        total += w
        if len(mst) == g.n - 1:
            break

if len(mst) != g.n - 1:
    return None, math.inf
return mst, total
```

โค้ดทำงานยังไง (ต่อ)

- **if rank[ra] < rank[rb]:** ถ้ากลุ่มของ ra มี rank ต่ำกว่า rb
- **parent[ra] = rb** ให้ราก ra ไปอยู่ใต้ rb (เชื่อม group เล็กเข้ากับ group ใหญ่)
- **elif rank[ra] > rank[rb]:** กรณีตรงข้าม: ถ้า ra มี rank สูงกว่า
- **parent[rb] = ra** ให้ rb มาอยู่ใต้ ra
- **else:** ถ้า rank เท่ากัน
- **parent[rb] = ra** เลือกให้ rb มาอยู่ใต้ ra (จะเลือกฝั่งไหนเป็นรากก็ได้)
- **rank[ra] += 1** เพิ่ม rank ของรากใหม่ ra เพราะต้นไม้สูงขึ้นหนึ่งระดับ
- **return True** รวมกลุ่มสำเร็จ แปลว่าการใช้เส้นระหว่าง a และ b “ไม่ทำให้เกิด cycle” สามารถใช้เส้นนี้ใน MST ได้
- ส่วนหลักของ **Kruskal mst = []** ลิสต์เก็บเส้นทั้งหมดที่ถูกเลือกให้เป็นส่วนหนึ่งของ MST **total = 0** ตัวแปรเก็บผลรวมของน้ำหนักเส้นทั้งหมดใน MST
- **for w, u, v in sorted(g.edges):** วนดูทุกเส้นในกราฟ **g.edges** โดยเรียงลำดับก่อนด้วย **sorted(...)** ให้เส้นน้ำหนักน้อยที่สุดมาก่อน แต่ละเส้นอยู่ในรูป (w, u, v) คือเส้นจาก u → v น้ำหนัก w
- **if union(u, v):** พยายามรวมกลุ่มของโหนด u และ ถ้า union คืนค่า True แสดงว่าเดิมทีสองโหนดยังอยู่คนละกลุ่ม → เพิ่มเส้นนี้ไม่ทำให้เกิด cycle ถ้าคืนค่า False แสดงว่าโหนดทั้งสองอยู่กลุ่มเดียวกันแล้ว →ข้ามเส้นนี้
- **mst.append((u, v, w))** ถ้า union สำเร็จ เพิ่มเส้น (u, v, w) ลงใน MST
- **total += w** เพิ่มค่าน้ำหนักของเส้นนี้เข้ากับผลรวม MST
- **if len(mst) == g.n - 1:** ถ้าจำนวนเส้นใน mst ครบ จำนวนโหนด - 1 แล้ว แสดงว่า MST สมบูรณ์ สามารถหยุดเลือกเส้นเพิ่มเติมได้
- **break** ออกจากลูปทันที ไม่ต้องพิจารณาเส้นที่เหลือ
- **if len(mst) != g.n - 1:** หลังจากจบลูป ถ้าจำนวนเส้นใน MST ยังไม่ถึง n - 1
- **return None, math.inf** แสดงว่ากราฟไม่เชื่อมถึงกันทุกโหนด (ไม่สามารถสร้าง MST ครบทุกโหนดได้)
- **return mst, total** มิฉะนั้นคืนค่า mst : รายชื่อเส้นใน Minimum Spanning Tree total : ผลรวมของน้ำหนักเส้นทั้งหมดใน MST

**ส่วนที่เหลือในโค้ดเป็นส่วนของ GUI
และการแสดงผล
ลิง github**

<https://github.com/Taiga0727/Graph-algorithm-visualizer>

จัดทำโดย

68090500414 phasit

68090500427 atilak

68090500406 chitsanuphong